



Analyse de malwares et sécurisation des données

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction à l'analyse de malwares : taxonomie et cycle de vie

Analyse de malwares et sécurisation des données

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Qu'est-ce qu'un malware ?

Un **malware** (contraction de *malicious software*, logiciel malveillant) désigne tout programme conçu intentionnellement pour nuire à un système informatique, à ses données ou à ses utilisateurs, sans le consentement légitime de la victime. Cette définition volontairement large recouvre des familles très différentes par leur mode de propagation, leur charge utile (*payload*) et leurs objectifs : espionnage, extorsion financière, sabotage, prise de contrôle à distance, ou simple nuisance.

L'**analyse de malwares** est la discipline qui consiste à examiner un échantillon suspect — de manière statique (sans l'exécuter) ou dynamique (en l'exécutant en environnement contrôlé) — afin de comprendre son fonctionnement, ses capacités, ses objectifs et les moyens de le détecter et de s'en protéger. C'est une discipline strictement défensive : son but est de produire de la connaissance utile à la protection (signatures, indicateurs de compromission, recommandations), jamais de produire ou d'améliorer du code offensif.

Ver (worm)

Un **ver** est un programme autonome capable de se propager de machine en machine sans intervention humaine, en général en exploitant une vulnérabilité réseau ou un service mal configuré. Contrairement au virus, il n'a pas besoin d'un fichier hôte : il se réplique lui-même et diffuse ses copies. Sa capacité de propagation automatique en fait historiquement l'une des familles capables de générer les incidents à plus grande échelle et les plus rapides à se répandre.

Cheval de Troie (trojan)

Un **cheval de Troie** se présente comme un logiciel légitime ou utile, mais dissimule une fonction malveillante cachée (vol de données, ouverture d'un accès distant, téléchargement d'autres charges utiles). À la différence du virus et du ver, il ne se réplique pas de lui-même : sa propagation dépend de l'ingénierie sociale (l'utilisateur est incité à l'installer volontairement). Les chevaux de Troie d'accès distant (*RAT — Remote Access Trojan*) donnent à l'attaquant un contrôle étendu de la machine infectée.

Ransomware (rançongiciel)

Un **ransomware** chiffre (ou prétend chiffrer) les données de la victime et exige le paiement d'une rançon en échange de la clé de déchiffrement. Cette famille illustre bien le lien entre analyse de malwares et sécurisation des données (chapitre 6) : la meilleure défense contre un ransomware n'est pas seulement la détection, mais aussi une politique de sauvegarde robuste et testée qui rend la rançon inutile.

Spyware (logiciel espion)

Un **spyware** collecte discrètement des informations sur l'utilisateur ou le système (frappes clavier, historique de navigation, identifiants) et les exfiltre vers l'attaquant. Un **keylogger** (enregistreur de frappe) en est une sous-catégorie fréquente. L'objectif est généralement la discrétion et la persistance plutôt que la destruction visible de données.

Rootkit

Un **rootkit** est un ensemble de techniques (souvent combinées à un autre malware) visant à dissimuler la présence d'un programme malveillant sur le système, en manipulant le système d'exploitation lui-même (interception d'appels système, modification de structures noyau) pour masquer des fichiers, des processus ou des connexions réseau à l'utilisateur et aux outils de sécurité classiques. Un rootkit fonctionnant au niveau noyau est

3. Cycle de vie d'une attaque malveillante

Comprendre l'analyse d'un échantillon isolé ne suffit pas : un malware s'inscrit généralement dans une chaîne d'attaque plus large. Un modèle classique, popularisé sous le nom de *cyber kill chain*, décompose cette chaîne en étapes successives (le vocabulaire varie selon les sources, l'idée générale reste stable) :

1. **Reconnaissance** : collecte d'informations sur la cible (technologies utilisées, employés, surface d'attaque).
2. **Armement (weaponization)** : préparation d'une charge malveillante, souvent couplée à un vecteur de livraison (document piégé, exploit).
3. **Livraison (delivery)** : transmission du malware à la cible (courriel de hameçonnage, site web compromis, clé USB, exploitation directe d'un service exposé).
4. **Exploitation** : déclenchement effectif du code malveillant, typiquement via l'exploitation d'une vulnérabilité logicielle ou l'exécution d'une pièce jointe par l'utilisateur.
5. **Installation** : mise en place de la persistance sur le système (démarrage automatique, tâche planifiée, service).
6. **Commande et contrôle (C2)** : établissement d'un canal de communication entre le malware et l'infrastructure de l'attaquant, permettant de recevoir des instructions ou d'exfiltrer des données.
7. **Actions sur objectifs** : réalisation de l'objectif final de l'attaquant (vol de données, chiffrement de rançon, sabotage, mouvement latéral vers d'autres systèmes).

Ce découpage a une conséquence pratique importante pour la défense : une interruption de la chaîne à n'importe quelle étape suffit à empêcher l'attaquant d'atteindre son objectif. C'est le principe de la **défense en profondeur** appliqué au cycle d'attaque : il n'est pas nécessaire de bloquer parfaitement chaque étape individuellement pour être efficace globalement.

Le référentiel **MITRE ATT&CK®**, largement utilisé dans l'industrie et repris au chapitre 5, formalise ce type de démarche de façon plus détaillée, en cataloguant des tactiques (les objectifs, proches des étapes ci-dessus) et des techniques (les moyens concrets observés pour les atteindre), à partir d'observations documentées de campagnes réelles.

4. Panorama du domaine de l'analyse de malwares

L'analyse de malwares se pratique à plusieurs niveaux de profondeur, qui structurent le reste de ce module :

- **Analyse statique** (chapitre 2) : étude du fichier sans l'exécuter — hachage, chaînes de caractères, structure du binaire, désassemblage.
- **Analyse dynamique** (chapitre 3) : observation du comportement de l'échantillon lors de son exécution dans un environnement isolé et instrumenté (*sandbox*).
- **Rétro-ingénierie** (chapitre 4) : analyse approfondie du code assembleur pour comprendre précisément la logique interne d'un échantillon, au-delà de son comportement observable en boîte noire.
- **Détection et réponse** (chapitre 5) : traduction des résultats d'analyse en mécanismes de défense opérationnels (signatures, règles heuristiques, indicateurs de compromission) et en procédures de réponse à incident.

Approche	Exécution de l'échantillon ?	Risque opérationnel	Ce qu'elle révèle
Analyse statique	Non	Minimal	Structure du fichier, chaînes, indices superficiels
Analyse dynamique	Oui, en environnement isolé	Maîtrisé par l'isolation stricte	Comportement réellement observé à l'exécution
Rétro-ingénierie	Non (étude du code, pas exécution)	Minimal	Logique interne détaillée, au-delà du comportement observable

Ces approches sont complémentaires plutôt que concurrentes : une analyse professionnelle combine généralement une première passe statique rapide, une exécution en sandbox pour observer le comportement réel, puis, si nécessaire, une rétro-ingénierie ciblée sur les parties les plus intéressantes du code.

5. Cadre éthique et légal de l'analyse de malwares

L'analyse de malwares implique, par nature, de manipuler du code potentiellement dangereux. Ce module impose donc un cadre strict, qui doit être considéré comme un prérequis non négociable à toute activité pratique :

- **Isolation stricte** : tout échantillon (y compris les échantillons pédagogiques bénins utilisés en TP) doit être manipulé dans une machine virtuelle dédiée, sans partage de dossiers avec la machine hôte, avec une carte réseau désactivée ou isolée dans un réseau interne sans accès à Internet ni au réseau de l'établissement.
- **Autorisation explicite** : en dehors du cadre pédagogique strictement encadré de ce cours, l'analyse d'un échantillon suspect doit se faire avec une autorisation claire (par exemple dans le cadre d'une mission professionnelle de réponse à incident) et jamais sur un système de production sans accord préalable.
- **Aucune manipulation de malware réel dans ce cours** : tous les échantillons utilisés en TP sont des programmes fictifs ou bénins construits spécifiquement pour l'exercice, qui *simulent* des comportements suspects (écriture de fichiers, tentative de connexion réseau) sans jamais causer de dommage réel ni contenir de code malveillant fonctionnel.
- **Non-divulgation irresponsable** : dans un contexte professionnel réel, la découverte d'une vulnérabilité ou d'un incident suit un principe de divulgation responsable (informer les parties concernées avant toute publication), qui sera repris au chapitre 5 dans le contexte de la réponse à incident.
- **Cadre légal** : la détention, la création ou la diffusion de code malveillant sans cadre légal approprié est un délit dans la plupart des juridictions. Ce module ne fournit et ne fournira jamais de code malveillant fonctionnel, ni d'instructions opérationnelles permettant d'en créer.

À retenir

- Un malware se définit par son intentionnalité malveillante ; ses familles (virus, ver, cheval de Troie, ransomware, spyware, rootkit) se distinguent surtout par leur mode de propagation et leur comportement, et se combinent souvent dans un même échantillon réel.
- Une attaque malveillante s'inscrit typiquement dans un cycle de vie en plusieurs étapes (reconnaissance, livraison, exploitation, installation, commande et contrôle, actions sur objectifs) : interrompre une seule étape suffit à protéger la cible.
- L'analyse de malwares se décline en plusieurs approches complémentaires — statique, dynamique, rétro-ingénierie — combinées en pratique plutôt qu'utilisées isolément.
- Ce module et l'ensemble de ses TP reposent sur un cadre éthique strict : isolation systématique, autorisation explicite, et absence totale de manipulation de code malveillant réel.
- MITRE ATT&CK constitue un référentiel de référence pour structurer la connaissance des tactiques et techniques adverses ; il sera approfondi au chapitre 5.

Chapitre 2 — Analyse statique de malwares

Analyse de malwares et sécurisation des données

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe de l'analyse statique

L'**analyse statique** consiste à examiner un échantillon suspect sans jamais l'exécuter. C'est généralement la première étape d'une analyse de malware : elle est rapide, ne présente aucun risque d'exécution accidentelle de code malveillant, et permet souvent d'obtenir des informations exploitables (identification d'une famille connue, indicateurs de compromission) sans avoir à mettre en place un environnement d'exécution isolé.

Son inconvénient principal est qu'elle ne révèle que ce qui est visible dans le fichier lui-même : un malware bien obfusqué ou chiffré (*packé*) peut dissimuler l'essentiel de son comportement réel à une analyse purement statique, ce qui justifie de la compléter par l'analyse dynamique (chapitre 3).

2. Hachage et identification

La première étape d'une analyse statique est presque toujours le calcul d'une **empreinte cryptographique** (hachage) de l'échantillon, qui permet de l'identifier de façon unique et de vérifier s'il correspond à un échantillon déjà connu et documenté.

```
# Calcul de plusieurs empreintes standards d'un fichier
sha256sum echantillon.bin
md5sum echantillon.bin
sha1sum echantillon.bin
```

En pratique, le hachage SHA-256 est aujourd'hui la référence pour l'identification d'échantillons (le MD5 et le SHA-1 restent utilisés par certains outils historiques, mais présentent des faiblesses cryptographiques étudiées dans le module *Cryptanalyse*, sans que cela remette en cause leur usage comme simple identifiant, hors contexte de sécurité). Ce hachage peut ensuite être confronté à des bases de connaissances répertoriant des échantillons déjà analysés, afin de déterminer immédiatement si le fichier correspond à une menace connue et documentée.

Une limite importante : un hachage cryptographique change radicalement dès que le fichier est modifié d'un seul octet. Des variantes légèrement modifiées d'un même malware (recompilation, ajout d'un octet inoffensif) produisent donc des hachages complètement différents, ce qui a motivé le développement de techniques de **hachage flou** (*fuzzy hashing*), capables de détecter une similarité partielle entre deux fichiers proches sans être identiques.

3. Extraction de chaînes de caractères

Un exécutable contient souvent des chaînes de caractères en clair — messages d'erreur, noms de fonctions, adresses réseau, chemins de fichiers, clés de registre — qui donnent des indices précieux sur son comportement, même sans le désassembler.

```
# Extraction des chaînes imprimables d'au moins 6 caractères
strings -n 6 echantillon.bin | less

# Recherche ciblée d'indices réseau ou de persistance
strings -n 6 echantillon.bin | grep -Ei "http://|https://|.php|CurrentVersion\\\\\\Run"
```

Cette technique simple révèle fréquemment des éléments exploitables : une URL de serveur de commande et contrôle, un nom de mutex utilisé pour éviter une double infection, une clé de registre de persistance, ou même des commentaires laissés par le développeur du malware. Sa principale limite est que les auteurs de malwares plus sophistiqués chiffrent ou encodent délibérément ces chaînes pour échapper à ce type d'inspection rapide (voir la section sur l'obfuscation ci-dessous).

4. Identification du format et de la structure du binaire

Avant d'aller plus loin, il est essentiel d'identifier le type de fichier réel (indépendamment de son extension, qui peut être trompeuse) et la structure de son format exécutable.

```
# Identification du type de fichier par analyse de son en-tête (magic bytes)
file echantillon.bin
```

Les deux formats d'exécutables les plus courants sont :

- **PE (Portable Executable)** : format des exécutables Windows (.exe, .dll). Sa structure comprend un en-tête MS-DOS historique, un en-tête PE proprement dit, une table de sections (.text pour le code, .data pour les données, .rsrc pour les ressources), et une table d'importation qui liste les fonctions des bibliothèques système utilisées par le programme.
- **ELF (Executable and Linkable Format)** : format standard des exécutables sous Linux et de nombreux systèmes Unix, structuré de façon comparable (en-tête, table de sections, table des symboles).

```
# Lister l'en-tête et les sections d'un exécutable (tailles, permissions)
objdump -h echantillon.bin

# Afficher les bibliothèques dynamiques liées et les symboles importés d'un ELF
readelf -d echantillon.bin
```

Sous Windows, des outils équivalents (dumpbin /imports fourni avec les outils de compilation Microsoft, ou l'utilitaire libre PE-bear) donnent la même vue sur un fichier PE : la liste des bibliothèques (.dll) et des fonctions précises qu'un exécutable prévoit d'appeler.

La **table d'importation** d'un exécutable PE (ou la table des symboles dynamiques d'un ELF) est particulièrement instructive : elle liste les fonctions système que le programme prévoit d'appeler. Des importations telles que des fonctions de manipulation du registre, de création de processus distant, ou de chiffrement, constituent des indices — non des preuves à elles seules — sur les capacités potentielles du programme.

5. Désassemblage de base

Le **désassemblage** consiste à traduire le code machine binaire (une suite d'octets) en instructions assembleur lisibles par un humain. C'est une étape plus coûteuse en temps que les précédentes, mais qui permet une compréhension bien plus fine du comportement du programme.

```
# Désassemblage sommaire en syntaxe Intel avec un outil GNU standard
objdump -d -M intel echantillon.bin | less
```

En pratique professionnelle, l'analyse manuelle d'un désassemblage complet est rarement exhaustive : l'analyste cible d'abord les fonctions les plus suspectes (repérées via les chaînes de caractères ou la table d'importation), plutôt que de lire linéairement l'ensemble du code. Les outils de désassemblage interactifs les plus utilisés dans l'industrie et la recherche (IDA Pro, Ghidra — cette dernière étant un outil de la NSA publié en open source, ou encore radare2, également libre) offrent en plus une reconstruction du flux de contrôle sous forme de graphe, une identification automatique des fonctions, et parfois une décompilation partielle vers un pseudo-code proche du C, qui facilite grandement la lecture par rapport à l'assembleur brut. Ces bases de lecture d'un désassemblage sont approfondies au chapitre 4.

6. Limites de l'analyse statique : obfuscation et packing

Les auteurs de malwares cherchent activement à contourner l'analyse statique, principalement par deux familles de techniques :

- **Obfuscation** : transformation du code ou des données pour les rendre plus difficiles à lire sans changer leur comportement (chiffrement de chaînes de caractères, insertion de code mort, renommage systématique de symboles, aplatissage du flux de contrôle). L'objectif est de ralentir l'analyste humain et de tromper les outils d'analyse automatisée basés sur des motifs simples.
- **Packing (empaquetage)** : compression ou chiffrement de tout ou partie du code exécutable réel, qui n'est déchiffré en mémoire qu'au moment de l'exécution, par une petite routine de décompression (*stub*) placée au début du programme. Un exécutable packé présente, à l'analyse statique, une entropie élevée (données qui ressemblent à du bruit aléatoire) et une table d'importation très réduite, car la majorité des fonctions réellement utilisées ne sont résolues qu'au moment de l'exécution.

```
# L'entropie de Shannon par section aide à repérer un empaquetage
# (une entropie proche de 8 bits/octet sur une section de code est suspecte)
python3 -c "
import math, collections
donnees = open('echantillon.bin', 'rb').read()
freq = collections.Counter(donnees)
entropie = -sum((n / len(donnees)) * math.log2(n / len(donnees)) for n in freq.values())
print(f'Entropie : {entropie:.2f} bits/octet (maximum théorique : 8)')
"
```

Au-delà d'un calcul manuel, des outils dédiés comme **Detect It Easy (DIE)** ou l'historique **PEiD** automatisent ce diagnostic : ils calculent l'entropie de chaque section et reconnaissent, à partir d'une base de signatures, les empaqueteurs les plus répandus (UPX, par exemple, largement utilisé aussi bien par des logiciels légitimes que par des malwares peu sophistiqués). Une identification positive du packer utilisé permet souvent de le désempaqueter automatiquement avec l'outil correspondant, retrouvant ainsi le code original avant de reprendre l'analyse statique.

Un packing détecté (par exemple une entropie anormalement élevée, ou la signature d'un empaqueteur connu) est en soi un indicateur utile, mais il ne dit rien du comportement réel du programme une fois déballé : c'est précisément la limite fondamentale de l'analyse purement statique face à ce type de protection. La suite logique consiste alors à observer le comportement du programme au moment de son exécution — c'est l'objet de l'analyse dynamique, présentée au chapitre suivant.

7. Méthodologie type d'une analyse statique

1. Calculer les empreintes cryptographiques de l'échantillon et les confronter à des sources documentées.
2. Identifier le type de fichier réel et sa structure (PE, ELF, script, document).
3. Extraire et examiner les chaînes de caractères pour repérer des indices immédiats (URL, chemins, clés de registre).
4. Examiner la table d'importation pour évaluer les capacités potentielles du programme.
5. Évaluer l'entropie et rechercher des signes d'obfuscation ou de packing.
6. Si nécessaire et pertinent, procéder à un désassemblage ciblé des fonctions les plus suspectes.
7. Documenter systématiquement les observations, même partielles : elles orienteront l'analyse dynamique.

À retenir

- L'analyse statique examine un échantillon sans l'exécuter : rapide et sans risque d'exécution accidentelle, mais limitée face à un code obfusqué ou packé.
- Le hachage cryptographique (SHA-256 en priorité) sert d'identifiant unique et permet de confronter un échantillon à des bases de connaissances existantes.
- L'extraction de chaînes de caractères et l'examen de la table d'importation fournissent des indices rapides et souvent très informatifs sur les capacités d'un programme.
- Les formats PE (Windows) et ELF (Linux) partagent une logique commune : en-tête, sections, table des symboles/importations.
- Le désassemblage traduit le code machine en assembleur lisible ; les outils modernes (Ghidra, IDA, radare2) facilitent cette lecture par une reconstruction du flux de contrôle et une décompilation partielle.
- Une entropie élevée et une table d'importation anormalement réduite sont des indices classiques de packing, qui limite fortement la portée de l'analyse statique seule.

Chapitre 3 — Analyse dynamique et sandboxing

Analyse de malwares et sécurisation des données

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe de l'analyse dynamique

L'**analyse dynamique** consiste à exécuter un échantillon suspect dans un environnement contrôlé et instrumenté, afin d'observer directement son comportement réel plutôt que de le déduire de sa seule structure statique. Elle répond directement à la limite principale du chapitre précédent : un code packé ou obfusqué, illisible à l'analyse statique, doit nécessairement se déchiffrer et révéler son comportement réel en mémoire au moment de son exécution pour pouvoir fonctionner.

Cette approche est complémentaire, non substitutive, à l'analyse statique : elle observe *ce que fait* le programme lors d'une exécution donnée, mais ne garantit pas d'avoir observé *tous* les comportements possibles (un malware peut, par exemple, ne déclencher sa charge malveillante que sous certaines conditions précises — une date donnée, la présence d'un fichier particulier — qui ne sont pas nécessairement réunies pendant l'observation).

2. Exigence absolue d'isolation

Exécuter un échantillon suspect, même pédagogique, exige une isolation stricte de l'environnement d'analyse. Ce principe est non négociable et s'applique à l'ensemble des TP de ce module :

- **Machine virtuelle dédiée** : l'analyse s'effectue dans une VM créée spécifiquement pour cet usage, jamais sur la machine physique de l'analyste.
- **Réseau isolé ou désactivé** : la carte réseau de la VM est soit désactivée, soit connectée à un réseau interne isolé (*host-only*), sans accès à Internet ni au réseau de l'établissement, afin d'éviter toute communication réelle avec une éventuelle infrastructure malveillante ou toute propagation accidentelle.
- **Aucun dossier partagé actif** pendant l'exécution de l'échantillon, afin d'éviter qu'un comportement inattendu n'affecte la machine hôte.
- **Instantanés (snapshots)** : un instantané propre de la VM est pris avant chaque analyse, permettant de revenir instantanément à un état sain après l'exécution, sans avoir à réinstaller le système.
- **Machine jetable** : une VM ayant servi à l'exécution d'un échantillon, même pédagogique, ne doit jamais être remise en usage courant sans être restaurée à un instantané sain ou recréée intégralement.

Une fois l'environnement isolé mis en place, l'analyse dynamique consiste à instrumenter le système pour observer plusieurs catégories d'activité pendant l'exécution de l'échantillon :

Appels système et API

L'interception des appels que le programme effectue auprès du système d'exploitation (création de processus, ouverture de fichiers, manipulation du registre sous Windows) constitue la source d'information la plus riche. Des outils de surveillance dédiés permettent d'enregistrer, en temps réel, la séquence complète de ces appels sans modifier le code du programme observé.

Exemple simplifié d'une trace comportementale (extrait pédagogique, format illustratif) :

```
CreateFileA("C:\\Users\\...\\Documents\\rapport.docx", GENERIC_READ)
RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\...\\Run")
RegSetValueExA(..., "UpdateService", "C:\\...\\echantillon.exe")
CreateProcessA(NULL, "cmd.exe /c ...", ...)
```

Une telle trace révèle directement des comportements que l'analyse statique n'aurait pu que soupçonner : ici, un accès à un document utilisateur, la modification d'une clé de démarrage automatique (persistance), et la création d'un nouveau processus.

Activité réseau

Même en environnement isolé sans accès Internet réel, on peut observer les *tentatives* de communication réseau du programme (résolutions DNS demandées, adresses IP et ports contactés, protocole utilisé) grâce à un réseau simulé ou à des services factices répondant aux requêtes sans exposer de véritable infrastructure. Cette technique, dite d'**émulation de services réseau**, permet de recueillir des indicateurs (domaines, adresses) sans jamais laisser l'échantillon atteindre un serveur réel.

Système de fichiers et registre

La comparaison d'un instantané du système de fichiers (et, sous Windows, du registre) avant et après l'exécution permet d'identifier précisément les fichiers créés, modifiés ou supprimés, ainsi que les clés de registre ajoutées ou modifiées — une méthode simple mais efficace pour repérer des mécanismes de persistance ou des traces laissées par le programme.

4. Détection d'évasion de sandbox par les malwares

Les auteurs de malwares sophistiqués savent que leurs échantillons sont susceptibles d'être analysés en environnement virtualisé, et intègrent parfois des mécanismes de **détection d'environnement d'analyse** (*sandbox evasion* ou *anti-analysis*) pour adapter leur comportement en conséquence — généralement en restant inactifs ou en n'affichant qu'un comportement inoffensif s'ils détectent qu'ils sont observés :

- **Détection de virtualisation** : recherche d'artefacts caractéristiques d'un hyperviseur (identifiants matériels spécifiques, pilotes ou processus associés aux outils de virtualisation courants, valeurs particulières renvoyées par certaines instructions processeur).
- **Détection d'environnement d'analyse** : recherche d'outils de surveillance en cours d'exécution, de noms de processus ou de fenêtres associés à des outils d'analyse connus.
- **Temporisation (sleep/delay)** : introduction de délais d'attente avant le déclenchement du comportement malveillant, dans l'espoir que la fenêtre d'observation automatisée (souvent limitée dans le temps) se termine avant l'activation réelle.
- **Vérification de l'interaction humaine** : attente d'un mouvement de souris, d'un redémarrage, ou d'une interaction utilisateur typique, absente dans une exécution automatisée sans surveillance humaine.
- **Conditions d'activation contextuelles** : déclenchement subordonné à une date précise, à la présence de fichiers spécifiques, ou à une configuration réseau particulière.

Face à ces techniques, l'analyste dispose de plusieurs contre-mesures pédagogiquement intéressantes à connaître : configuration de la VM pour minimiser les artefacts de virtualisation détectables, prolongation de la durée d'observation, simulation d'interactions utilisateur, ou avance artificielle de l'horloge système pour révéler un comportement temporisé. Ce jeu du chat et de la souris illustre bien la nature évolutive de l'analyse de malwares : chaque contre-mesure défensive peut, à terme, susciter une nouvelle technique d'évasion.

5. Bonnes pratiques d'isolation en pratique

Bonne pratique	Justification
VM dédiée, jamais la machine hôte	Éviter toute compromission réelle de l'environnement de l'analyste
Réseau host-only ou désactivé	Empêcher toute communication réelle avec une infrastructure malveillante
Instantané avant chaque analyse	Revenir à un état sain instantanément, sans réinstallation
Restauration systématique après usage	Éviter la persistance de tout artefact entre deux analyses
Documentation de chaque comportement observé	Constituer une base d'indicateurs de compromission réutilisable (chapitre 5)
Aucune exécution en dehors de l'environnement isolé	Principe non négociable, y compris pour un échantillon jugé « probablement inoffensif »

6. Complémentarité avec l'analyse statique

Une méthodologie professionnelle typique enchaîne les deux approches : l'analyse statique (chapitre 2) oriente l'analyse dynamique en identifiant les fonctions ou chaînes suspectes à surveiller en priorité, tandis que l'analyse dynamique révèle le comportement réel, y compris pour du code packé ou obfusqué illisible statiquement. Les observations comportementales issues de ce chapitre alimentent directement la production d'indicateurs de compromission et de règles de détection, développée au chapitre 5.

À retenir

- L'analyse dynamique observe le comportement réel d'un échantillon en l'exécutant dans un environnement isolé, complétant les limites de l'analyse statique face à l'obfuscation et au packing.
- L'isolation stricte (VM dédiée, réseau désactivé ou isolé, instantanés, restauration systématique) est un prérequis non négociable, y compris pour des échantillons pédagogiques bénins.
- La surveillance comportementale porte typiquement sur les appels système, l'activité réseau (simulée en environnement isolé) et les modifications du système de fichiers et du registre.
- Certains malwares intègrent des techniques d'évasion de sandbox (détection de virtualisation, temporisation, attente d'interaction humaine) pour dissimuler leur comportement réel lors d'une analyse automatisée.
- L'analyse dynamique n'observe qu'une exécution donnée : elle ne garantit pas d'avoir révélé tous les comportements possibles d'un échantillon, notamment ceux soumis à des conditions d'activation particulières.

Chapitre 4 — Techniques de rétro-ingénierie de base

Analyse de malwares et sécurisation des données

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Cadre pédagogique et légal de ce chapitre

Ce chapitre introduit des notions élémentaires de **rétro-ingénierie** (*reverse engineering*), c'est-à-dire l'analyse d'un programme compilé pour en comprendre le fonctionnement interne, sans disposer de son code source. Ces notions sont enseignées ici dans un unique but défensif : elles permettent à un analyste de comprendre plus précisément un comportement déjà observé (par l'analyse statique ou dynamique) et de produire des indicateurs de détection fiables. Comme pour les chapitres précédents, tout exercice pratique s'effectue exclusivement sur des programmes pédagogiques bénins, en environnement isolé, jamais sur du code malveillant réel.

Un programme compilé en langage machine est une suite d'instructions binaires interprétées directement par le processeur. L'**assembleur** est la représentation textuelle lisible par un humain de ces instructions machine : chaque instruction assembleur correspond directement à une instruction machine (à la différence d'un langage de haut niveau comme le C, qui est traduit en plusieurs instructions machine par le compilateur).

L'architecture **x86** (et son extension 64 bits, x86-64) reste la plus répandue sur les postes de travail et serveurs, ce qui en fait la référence pédagogique la plus utile pour une première initiation.

Registres principaux (x86-64)

Les registres sont de petites zones de mémoire directement intégrées au processeur, utilisées pour les calculs et le passage de données entre instructions :

Registre	Rôle usuel
RAX	Registre accumulateur ; contient souvent la valeur de retour d'une fonction
RBX, RCX, RDX	Registres généraux, utilisés selon les conventions d'appel et les besoins du compilateur
RSP	Pointeur de pile (<i>stack pointer</i>) : sommet de la pile d'exécution courante
RBP	Pointeur de base de pile (<i>base pointer</i>) : repère fixe du cadre de pile de la fonction courante
RIP	Pointeur d'instruction (<i>instruction pointer</i>) : adresse de la prochaine instruction à exécuter

Instructions assembleur de base

```
; Extrait pédagogique en syntaxe Intel, à but strictement illustratif
mov  rax, 5      ; charge la valeur 5 dans le registre RAX
add  rax, rbx    ; RAX <- RAX + RBX
cmp  rax, 10     ; compare RAX à 10 (met à jour les indicateurs)
je   etiquette_egal ; saut conditionnel si l'égalité est vérifiée
call fonction_x  ; appel de fonction (empile l'adresse de retour)
ret              ; retour de fonction (dépile l'adresse de retour)
```

Cette poignée d'instructions (déplacement de données `mov`, arithmétique `add/sub`, comparaison `cmp`, sauts conditionnels `je/jne/jg`, appel et retour de fonction `call/ret`) suffit à lire une part importante d'un désassemblage simple. L'objectif de ce chapitre n'est pas la maîtrise complète du

3. Structure d'un exécutable : notions générales PE et ELF

Comprendre la structure d'un exécutable, déjà abordée au chapitre 2 du point de vue de l'analyse statique, prend ici une importance pratique directe : c'est cette structure qui indique à l'outil de désassemblage où se trouve le code à analyser.

- Un exécutable **PE** (Windows) ou **ELF** (Linux) contient un en-tête décrivant le format du fichier, suivi d'une table de sections. La section `.text` (ou équivalent) contient le code exécutable proprement dit ; c'est elle qui est désassemblée.
- Le **point d'entrée** (*entry point*) indique l'adresse de la première instruction exécutée au lancement du programme. Dans un exécutable non packé, il correspond en général au début de la fonction principale du programme (ou d'une routine d'initialisation du runtime qui l'appelle). Dans un exécutable packé (chapitre 2), il pointe vers la petite routine de décompression, et non vers le code réel du programme.
- La **table d'importation** relie les appels de fonctions du code à des bibliothèques externes (système ou tierces) : un outil de désassemblage s'en sert pour afficher des noms de fonctions lisibles (par exemple une fonction de création de fichier) plutôt qu'une simple adresse mémoire.

4. Lecture d'un désassemblage simple

Reprenons un exemple pédagogique de fonction en C, puis son désassemblage simplifié, pour illustrer la démarche de lecture :

```
/* Fonction pédagogique en C, à but strictement illustratif */
int verifier_code(int code) {
    if (code == 1234) {
        return 1; /* code correct */
    }
    return 0; /* code incorrect */
}
```

```
; Désassemblage simplifié et annoté (syntaxe Intel, illustratif)
verifier_code:
    cmp     edi, 1234        ; compare l'argument (edi) à 1234
    jne     code_incorrect   ; si différent, saute vers le cas d'échec
    mov     eax, 1           ; sinon, valeur de retour = 1
    ret
code_incorrect:
    mov     eax, 0           ; valeur de retour = 0
    ret
```

La démarche de lecture typique consiste à repérer les **comparaisons** (`cmp`) et les **sauts conditionnels** qui les suivent, car ils traduisent directement la logique conditionnelle (`if/else`) du code source. Une constante comme `1234` apparaissant en dur dans une comparaison — ici un exemple pédagogique de « code d'accès » — est un motif classique à repérer : dans un contexte réel d'analyse défensive, une telle constante correspond souvent à une valeur significative (mot de passe codé en dur, identifiant de licence, condition d'activation).

5. Repérage de fonctions suspectes

Face à un désassemblage complet, souvent volumineux, l'analyste ne lit pas linéairement l'ensemble du code : il concentre son attention sur des indices qui orientent vers les fonctions les plus susceptibles de porter une logique intéressante :

- **Fonctions appelant des API sensibles** : une fonction qui appelle des primitives de manipulation du registre, de création de processus, ou de chiffrement mérite un examen prioritaire.
- **Fonctions à proximité de chaînes de caractères suspectes** identifiées lors de l'analyse statique (chapitre 2) : un outil de désassemblage permet généralement de retrouver, à partir d'une chaîne repérée, les fonctions qui la référencent.
- **Boucles et comparaisons répétées sur de petites constantes** : motif fréquent d'une routine de déchiffrement ou de décodage simple (par exemple un XOR répété avec une clé courte), potentiellement révélatrice d'une tentative d'obfuscation.
- **Fonctions rarement appelées ou atteintes seulement sous certaines conditions** : peuvent correspondre à une charge malveillante conditionnelle (déclenchement à une date donnée, par exemple), un point à mettre en relation avec les techniques d'évasion étudiées au chapitre 3.

6. Décompilation : un complément utile

Au-delà du simple désassemblage, des outils modernes proposent une **décompilation** partielle, qui reconstruit une approximation de code source de haut niveau (proche du C) à partir du code assembleur. Cette reconstruction n'est jamais parfaitement fidèle au code source original (certaines informations, comme les noms de variables ou les commentaires, sont irrémédiablement perdues à la compilation), mais elle accélère considérablement la compréhension d'une fonction complexe par rapport à la lecture de l'assembleur brut, et constitue aujourd'hui une pratique standard en analyse de malwares professionnelle.

Repris sur l'exemple de la section 4, un décompilateur produirait typiquement un pseudo-code de ce type à partir du même désassemblage :

```
/* Pseudo-code reconstruit automatiquement par un décompilateur
   (exemple illustratif, à partir du désassemblage de verifier_code) */
int verifier_code(int code) {
    if (code != 1234) {
        return 0;
    }
    return 1;
}
```

La comparaison avec le code source original de la section 4 illustre bien les limites de l'exercice : la logique et les valeurs numériques sont fidèlement reconstruites, mais le nom des variables (ici conservé par hasard dans cet exemple pédagogique simplifié), les commentaires (`/* code correct */`, `/* code incorrect */`) et parfois la forme exacte des conditions (un décompilateur peut, par exemple, inverser un test `==` en `!=` en réorganisant les branches) ne sont jamais garantis identiques à l'original. L'analyste doit donc toujours valider une hypothèse issue de la décompilation en la confrontant, si nécessaire, au désassemblage brut.

À retenir

- La rétro-ingénierie de base permet de comprendre précisément la logique d'un programme compilé, en complément de l'analyse statique et dynamique, dans un but strictement défensif.
- Les registres (`RAX`, `RSP`, `RIP`, etc.) et une poignée d'instructions clés (`mov`, `cmp`, sauts conditionnels, `call/ret`) suffisent à lire une part importante d'un désassemblage simple.
- La structure d'un exécutable (PE ou ELF) détermine où se trouve le code à désassembler et où se situe le point d'entrée du programme.
- La lecture d'un désassemblage se concentre sur les comparaisons et sauts conditionnels, qui traduisent la logique conditionnelle du code source original.
- Le repérage de fonctions suspectes s'appuie sur des indices (API sensibles, proximité de chaînes suspectes, motifs de boucle de déchiffrement) plutôt que sur une lecture linéaire exhaustive.
- La décompilation moderne (reconstruction d'un pseudo-code proche du C) accélère fortement l'analyse, sans jamais restituer parfaitement le code source d'origine.
- Ce chapitre, comme l'ensemble du module, s'exerce exclusivement sur des programmes pédagogiques bénins, jamais sur du code malveillant réel.

Chapitre 5 — Détection et réponse

Analyse de malwares et sécurisation des données

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. De l'analyse à la défense opérationnelle

Les chapitres précédents ont porté sur l'analyse d'un échantillon isolé. Ce chapitre fait le lien entre cette analyse et la défense d'un système d'information à l'échelle opérationnelle : comment détecter des malwares en général (connus ou inconnus), comment structurer la connaissance acquise sur les menaces, et comment organiser la réponse lorsqu'une compromission est identifiée.

2. Signatures, heuristiques et détection comportementale

Les logiciels de sécurité (antivirus, EDR) reposent sur plusieurs approches de détection, complémentaires et de maturité croissante :

Détection par signature

Une **signature** est un motif caractéristique (souvent dérivé d'un hachage ou d'une séquence d'octets spécifique) associé à un malware déjà connu et analysé. C'est la méthode de détection historique, rapide et fiable pour les menaces déjà répertoriées, mais par construction incapable de détecter un malware inédit (dit *zero-day*) ou une variante suffisamment modifiée pour échapper à la signature exacte.

Détection heuristique

L'**heuristique** généralise la signature en recherchant des motifs suspects plus larges (structure de code caractéristique, séquences d'instructions typiques d'un packer connu, combinaisons d'appels système inhabituelles) sans exiger une correspondance exacte. Elle permet de détecter des variantes non répertoriées d'une famille connue, au prix d'un risque plus élevé de **faux positifs** (un programme légitime signalé à tort comme malveillant).

Détection comportementale

La **détection comportementale** observe l'activité d'un programme en cours d'exécution (ou son historique récent) et la compare à des modèles de comportement normal ou anormal, indépendamment de sa signature statique. Elle peut détecter des menaces totalement inconnues, à condition que leur comportement s'écarte suffisamment du profil attendu. C'est le prolongement naturel, à l'échelle d'un parc de machines, de l'analyse dynamique en sandbox étudiée au chapitre 3.

Approche	Détecte les menaces inconnues ?	Risque de faux positifs	Coût de calcul
Signature	Non	Faible	Faible
Heuristique	Partiellement	Modéré	Modéré
Comportementale	Oui, en principe	Plus élevé	Plus élevé

En pratique, les solutions de sécurité modernes combinent systématiquement ces trois approches, chacune compensant les limites des autres.

3. EDR — Endpoint Detection and Response

L'**EDR** (*Endpoint Detection and Response*) désigne une catégorie d'outils de sécurité installés sur les postes de travail et serveurs (les *endpoints*), qui combinent :

- une **collecte continue de télémétrie** détaillée sur l'activité du système (processus créés, connexions réseau, accès fichiers, modifications de registre) — une forme de surveillance comportementale permanente, à l'échelle de l'ensemble du parc plutôt que d'un seul échantillon en sandbox ;
- une **détection** combinant signatures, heuristiques et analyse comportementale sur cette télémétrie ;
- des **capacités de réponse** intégrées, permettant d'isoler une machine compromise du réseau, de terminer un processus malveillant, ou de restaurer un fichier modifié, directement depuis la console de gestion.

L'EDR se distingue d'un antivirus traditionnel par sa visibilité beaucoup plus large (historique détaillé et interrogeable de l'activité, pas seulement un verdict binaire « sain/infecté ») et par sa capacité à détecter des attaques qui progressent lentement et discrètement (mouvement latéral, exfiltration progressive), en corrélant des événements individuellement anodins mais collectivement suspects.

4. Indicateurs de compromission (IOC)

Un **indicateur de compromission** (*Indicator of Compromise*, IOC) est un artefact technique observable qui signale, avec un degré de confiance variable, la présence ou le passage d'une menace sur un système. Les IOC constituent le langage commun par lequel les résultats d'une analyse de malware (chapitres 2 à 4) sont transformés en éléments de détection réutilisables.

Exemples de catégories d'IOC courantes :

Catégorie	Exemples
Réseau	Adresse IP ou domaine de commande et contrôle, motif d'URL, empreinte de certificat TLS
Fichier	Hachage SHA-256 d'un échantillon, nom de fichier caractéristique, chemin de dépôt typique
Registre / système	Clé de registre de persistance, nom de service ou de tâche planifiée créé par le malware
Comportemental	Séquence d'appels système caractéristique, nom de mutex utilisé pour éviter une double infection

Les IOC ont une durée de vie limitée : un attaquant change facilement une adresse IP ou recompile un échantillon pour changer son hachage. C'est pourquoi les IOC les plus robustes et durables sont ceux qui portent sur des éléments comportementaux difficiles à modifier sans reconcevoir substantiellement l'attaque — une idée centrale du cadre MITRE ATT&CK présenté ci-dessous.

5. MITRE ATT&CK® comme cadre de référence

MITRE ATT&CK® (*Adversarial Tactics, Techniques, and Common Knowledge*) est une base de connaissances publique et largement adoptée dans l'industrie, qui catalogue les **tactiques** (les objectifs poursuivis par un attaquant, comme l'accès initial, la persistance ou l'exfiltration) et les **techniques** (les moyens concrets observés pour atteindre ces objectifs), à partir de l'observation documentée de campagnes réelles.

Sa structure matricielle croise :

- les **tactiques**, qui reprennent et affinent la logique du cycle de vie d'une attaque déjà présentée au chapitre 1 (reconnaissance, accès initial, exécution, persistance, élévation de privilèges, contournement de défense, accès aux identifiants, découverte, mouvement latéral, collecte, exfiltration, impact) ;
- les **techniques** (et sous-techniques), qui décrivent des moyens précis d'atteindre chaque tactique (par exemple, plusieurs techniques distinctes permettent d'obtenir une persistance sur un système Windows).

L'intérêt pédagogique et opérationnel d'ATT&CK est double : il fournit un **vocabulaire commun** partagé par toute l'industrie pour décrire une attaque de façon précise et comparable d'un rapport à l'autre, et il permet à une organisation d'évaluer sa **couverture de détection** (quelles techniques du référentiel ses outils actuels permettent-ils réellement de détecter) de façon structurée plutôt qu'ad hoc.

6. Notions de réponse à incident

Lorsqu'une compromission est détectée (ou fortement suspectée), la réponse suit généralement un cycle en plusieurs phases, formalisé notamment par le NIST dans son guide de gestion des incidents :

1. **Préparation** : mise en place en amont des outils, procédures et compétences nécessaires (avant tout incident).
2. **Détection et analyse** : identification de l'incident, confirmation de sa réalité, évaluation de son ampleur — c'est ici que s'appliquent directement les techniques d'analyse de malware des chapitres précédents.
3. **Confinement (containment)** : limitation de la propagation de l'incident, par exemple par l'isolation réseau d'une machine compromise (fonctionnalité typique d'un EDR, section 3).
4. **Éradication** : suppression de la cause de l'incident (retrait du malware, fermeture de la vulnérabilité exploitée).
5. **Restauration (recovery)** : retour à un fonctionnement normal, notamment à partir de sauvegardes saines (lien direct avec le chapitre 6).
6. **Enseignements tirés (lessons learned)** : analyse rétrospective de l'incident pour améliorer les défenses et les procédures futures, y compris la mise à jour des indicateurs de compromission et de la couverture de détection ATT&CK.

Cette dernière phase illustre la nature cyclique de la sécurité opérationnelle : chaque incident, correctement analysé, alimente la préparation face aux incidents suivants.

À retenir

- La détection combine trois approches complémentaires : signatures (rapides, limitées aux menaces connues), heuristiques (variantes non répertoriées, plus de faux positifs) et comportementale (menaces inconnues, coût plus élevé).
- L'EDR étend la détection classique par une télémétrie continue et détaillée à l'échelle du parc, associée à des capacités de réponse intégrées (isolation, terminaison de processus).
- Un indicateur de compromission (IOC) traduit un résultat d'analyse en élément de détection réutilisable ; les IOC comportementaux sont plus durables que les IOC techniques facilement modifiables (IP, hachage).
- MITRE ATT&CK offre un vocabulaire commun et structuré (tactiques et techniques) pour décrire les attaques et évaluer la couverture de détection d'une organisation.
- La réponse à incident suit un cycle formalisé (préparation, détection, confinement, éradication, restauration, enseignements tirés) qui boucle directement sur l'analyse de malware et la sécurisation des données, objet du chapitre suivant.

Chapitre 6 — Sécurisation des données

Analyse de malwares et sécurisation des données

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi ce chapitre conclut le module

Les chapitres précédents ont porté sur la détection et la compréhension des menaces (malwares, techniques d'attaque). Ce dernier chapitre déplace le regard vers l'objectif final que ces menaces visent le plus souvent : les **données** elles-mêmes. Un système parfaitement protégé contre les malwares mais dont les données sont mal classifiées, mal chiffrées ou non sauvegardées reste vulnérable, en particulier face à des menaces comme le ransomware (chapitre 1), dont l'objectif porte précisément sur la disponibilité et la confidentialité des données. La sécurisation des données est donc le complément indispensable, et non un sujet indépendant, de tout ce qui précède.

2. Chiffrement au repos et en transit

Le **chiffrement des données** constitue la protection technique fondamentale de leur confidentialité, à distinguer selon l'état des données concernées :

- **Chiffrement au repos (data at rest)** : protège les données stockées durablement (disque, base de données, sauvegarde), contre un accès non autorisé en cas de vol physique du support ou de compromission du système de stockage. Il se décline typiquement au niveau du disque entier (chiffrement intégral), d'un système de fichiers, ou au niveau applicatif (chiffrement d'un champ particulier avant son enregistrement en base).
- **Chiffrement en transit (data in transit)** : protège les données pendant leur transmission sur un réseau, contre l'interception. C'est le rôle de protocoles comme TLS (étudié en détail dans le module *Cryptographie*), désormais quasi systématique pour toute communication sensible sur Internet.

```
# Exemple pédagogique : chiffrement symétrique d'un fichier au repos avec OpenSSL
openssl enc -aes-256-cbc -pbkdf2 -salt -in donnees.csv -out donnees.csv.enc

# Déchiffrement correspondant
openssl enc -d -aes-256-cbc -pbkdf2 -in donnees.csv.enc -out donnees.csv
```

Un principe souvent négligé mérite d'être souligné : le chiffrement protège la confidentialité des données, mais ne remplace ni un contrôle d'accès rigoureux (une donnée déchiffrée par un utilisateur légitime redevient lisible), ni une gestion sérieuse des clés — une clé de chiffrement mal protégée annule l'essentiel du bénéfice du chiffrement lui-même.

3. Classification des données selon leur sensibilité

La **classification des données** consiste à catégoriser les informations d'une organisation selon leur niveau de sensibilité, afin d'appliquer des mesures de protection proportionnées — ni excessives (coûteuses et contre-productives sur des données peu sensibles), ni insuffisantes (risquées sur des données critiques). Un schéma de classification simple et couramment rencontré distingue :

Niveau	Exemple	Mesures typiques associées
Public	Communication officielle, documentation publique	Aucune restriction particulière
Interne	Documents de travail internes non sensibles	Accès réservé aux membres de l'organisation
Confidentiel	Données financières, contrats, données personnelles	Chiffrement, contrôle d'accès strict, journalisation des accès
Hautement confidentiel / secret	Secrets industriels, données de santé, identifiants d'accès critiques	Chiffrement renforcé, accès restreint et tracé, procédures d'approbation

Cette classification n'a de valeur que si elle est effectivement appliquée : elle doit être portée par une politique claire (qui décide du niveau d'un document, comment il est marqué et communiqué) et par des outils qui appliquent automatiquement les mesures associées à chaque niveau, plutôt que de reposer uniquement sur la discipline individuelle des utilisateurs.

4. Contrôle d'accès

Le **contrôle d'accès** régit qui peut consulter, modifier ou supprimer une donnée donnée, en s'appuyant en général sur deux principes complémentaires :

- **Principe du moindre privilège** : chaque utilisateur ou processus ne dispose que des droits strictement nécessaires à l'exercice de sa fonction, ni plus. Ce principe limite mécaniquement l'ampleur des dégâts en cas de compte compromis — un lien direct avec la notion de confinement vue au chapitre 5.
- **Séparation des tâches** : une action sensible (par exemple, une modification critique de configuration) nécessite l'intervention coordonnée de plusieurs personnes distinctes, réduisant le risque d'abus ou d'erreur isolée.

Les modèles courants de contrôle d'accès incluent le contrôle d'accès basé sur les rôles (*Role-Based Access Control*, RBAC), qui attribue des droits à des rôles fonctionnels plutôt qu'individuellement à chaque utilisateur, facilitant la gestion à grande échelle et l'audit des droits accordés.

5. Prévention de la perte de données (DLP)

Les solutions de **prévention de la perte de données** (*Data Loss Prevention*, DLP) visent à détecter et bloquer les transferts non autorisés de données sensibles, qu'ils soient malveillants (exfiltration par un attaquant ou un employé malintentionné) ou accidentels (erreur humaine, envoi par erreur d'un fichier sensible). Elles s'appuient typiquement sur :

- l'**inspection de contenu**, capable de reconnaître des motifs caractéristiques de données sensibles (numéros de documents d'identité, structures de données personnelles, mots-clés associés à une classification donnée) ;
- le **contrôle des canaux de sortie** (messagerie, périphériques amovibles, services de stockage en ligne, impression), sur lesquels des règles de blocage ou d'alerte peuvent être appliquées ;
- l'intégration avec la classification des données (section 3) : une règle DLP peut, par exemple, bloquer automatiquement l'envoi externe d'un document marqué « confidentiel ».

Le DLP illustre un principe déjà rencontré au chapitre 5 avec l'EDR : la valeur de l'outil dépend largement de la qualité de sa configuration et de son intégration avec les autres briques de sécurité (classification, contrôle d'accès), plutôt que de sa seule installation.

6. Sauvegardes et plans de reprise

Face à une menace comme le ransomware (chapitre 1) ou à un incident de disponibilité de toute autre nature (panne, erreur humaine, sinistre), la **sauvegarde** des données constitue la dernière ligne de défense, souvent la plus décisive en pratique.

Une politique de sauvegarde robuste répond à quelques principes largement reconnus dans l'industrie, souvent résumés par la règle dite **3-2-1** :

- **3 copies** des données au minimum (l'original plus deux copies) ;
- sur **2 supports différents** (par exemple disque local et stockage distant), pour limiter le risque qu'une même défaillance matérielle affecte toutes les copies ;
- dont **1 copie hors site** (ou déconnectée), pour se prémunir d'un sinistre physique localisé ou, précisément, d'un ransomware qui chiffrerait des sauvegardes restées connectées et accessibles en écriture depuis le système compromis.

Deux indicateurs structurent un **plan de reprise d'activité** (*Disaster Recovery Plan*) :

- le **RPO** (*Recovery Point Objective*) : quantité maximale de données qu'une organisation accepte de perdre, exprimée en durée depuis la dernière sauvegarde valide ;
- le **RTO** (*Recovery Time Objective*) : durée maximale acceptable d'indisponibilité avant restauration complète du service.

Une sauvegarde n'a de valeur que si sa **restauration** a été testée régulièrement : une sauvegarde jamais restaurée en conditions réelles présente un risque non négligeable de s'avérer inutilisable au moment critique, un constat récurrent dans les retours d'expérience de réponse à incident.

7. Lien avec le RGPD et les référentiels de protection des données

De nombreuses juridictions imposent aujourd'hui des obligations légales spécifiques concernant la protection des données à caractère personnel, dont le **Règlement général sur la protection des données** (RGPD) de l'Union européenne constitue l'exemple le plus connu et le plus souvent cité comme référence internationale. Sans entrer dans le détail juridique précis, qui dépasse le cadre de ce module et varie selon les juridictions, quelques principes généraux qu'il porte recoupent directement les notions techniques de ce chapitre :

- **minimisation** : ne collecter et ne conserver que les données réellement nécessaires, ce qui réduit d'autant la surface exposée en cas de compromission ;
- **sécurité par défaut et dès la conception** (*privacy by design*), qui recoupe les principes de classification et de contrôle d'accès vus dans ce chapitre ;
- **notification des violations de données** dans des délais contraints, ce qui fait le lien direct avec les procédures de réponse à incident du chapitre 5 ;
- **droits des personnes concernées** (accès, rectification, effacement), qui supposent une bonne maîtrise de l'inventaire et de la classification des données détenues.

Au-delà du cadre européen, des référentiels techniques tels que les publications spéciales du NIST sur la sécurité de l'information et l'assainissement des supports de stockage fournissent des recommandations opérationnelles reconnues internationalement pour la mise en œuvre technique de ces principes.

À retenir

- Le chiffrement au repos et en transit protège la confidentialité des données, mais doit être complété par un contrôle d'accès rigoureux et une gestion sérieuse des clés.
- La classification des données par sensibilité permet d'appliquer des mesures de protection proportionnées, à condition d'être effectivement portée par une politique claire et des outils qui l'appliquent.
- Le contrôle d'accès (moindre privilège, séparation des tâches, RBAC) et les solutions DLP limitent respectivement l'ampleur d'une compromission et le risque d'exfiltration, malveillante ou accidentelle.
- La règle 3-2-1 (trois copies, deux supports, une hors site) et le test régulier de restauration constituent la meilleure défense pratique contre les menaces visant la disponibilité des données, notamment le ransomware.
- Le RPO et le RTO formalisent, respectivement, la perte de données et l'indisponibilité maximales acceptables par une organisation.
- Le RGPD et les référentiels comme les publications du NIST recourent, au niveau des principes généraux, l'ensemble des notions techniques abordées dans ce chapitre : minimisation, sécurité dès la conception, notification d'incident, maîtrise de l'inventaire des données.